Student Record Search System

Description

Student Record Search System is a menu-driven C++ program that stores student records in a Binary Search Tree (BST). Each record contains an enrolment ID, name, and department. The enrolment ID is used as the key for arranging records in the tree.

The program provides insertion, searching, deletion, sorted record display, tree traversals, efficiency comparison, and mandatory test cases.

Features

• Insert a new student record
• Search a student record by enrolment ID
• Delete a student record
• Display records in sorted order using inorder traversal
• Display preorder and postorder traversals
• Compare balanced-looking and skewed BST search efficiency
• Execute mandatory test cases

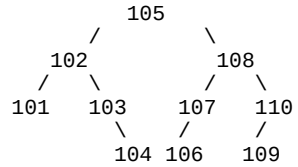
BST Used

The program uses a Binary Search Tree. For each node, IDs smaller than the node's ID are placed in the left subtree, while larger IDs are placed in the right subtree. Duplicate enrolment IDs are rejected.

Initial Student Records

Enrolment ID	Name	Department
105	Aarav Sharma	CSE
102	Diya Patel	ECE
108	Rohan Verma	ME
101	Ananya Gupta	CSE
103	Kabir Singh	CE
107	Isha Nair	ECE
110	Siddharth Rao	CSE
104	Meera Joshi	IT
106	Aditya Kumar	ME
109	Priya Reddy	IT

BST Structure



Menu Options

Choice	Action	Input required
1	Insert New Student Record	ID, name, department
2	Search Student Record by ID	Enrolment ID
3	Delete Student Record	Enrolment ID
4	Display Records (Inorder - Sorted)	None
5	Display Preorder & Postorder Traversals	None
6	Compare Balanced vs Skewed BST Efficiency	None
7	Run Mandatory Test Cases	None
8	Exit	None

Test Cases

#	Test	Input	Expected output	Result
1	Search existing record	105	Record found: Aarav Sharma (CSE); 1 comparison	PASS
2	Search missing record	999	Record not found; 3 comparisons	PASS
3	Duplicate key insertion	105	Duplicate ID error; record rejected	PASS
4	Leaf node deletion	101	Record deleted successfully	PASS
5	Single-child node deletion	110	Record deleted successfully (node has child 109)	PASS
6	Two-children node deletion	105	Record deleted successfully	PASS
7	Balanced-looking BST efficiency	Search 110	3 comparisons	PASS
8	Skewed BST efficiency	Search 110	7 comparisons	PASS

Complexity

Insertion, search, and deletion take O(h) time, where h is the height of the BST. For a balanced-looking tree, this is approximately O(log n); in a skewed tree, the worst case is O(n). Inorder, preorder, and postorder traversals take O(n) time. Storage for n nodes is O(n).

BY

Name: Chinmay Swami

Roll No.: 2556444

Course: B.Tech CSE